ПРИКЛАДНОЙ УРОВЕНЬ
Часть II

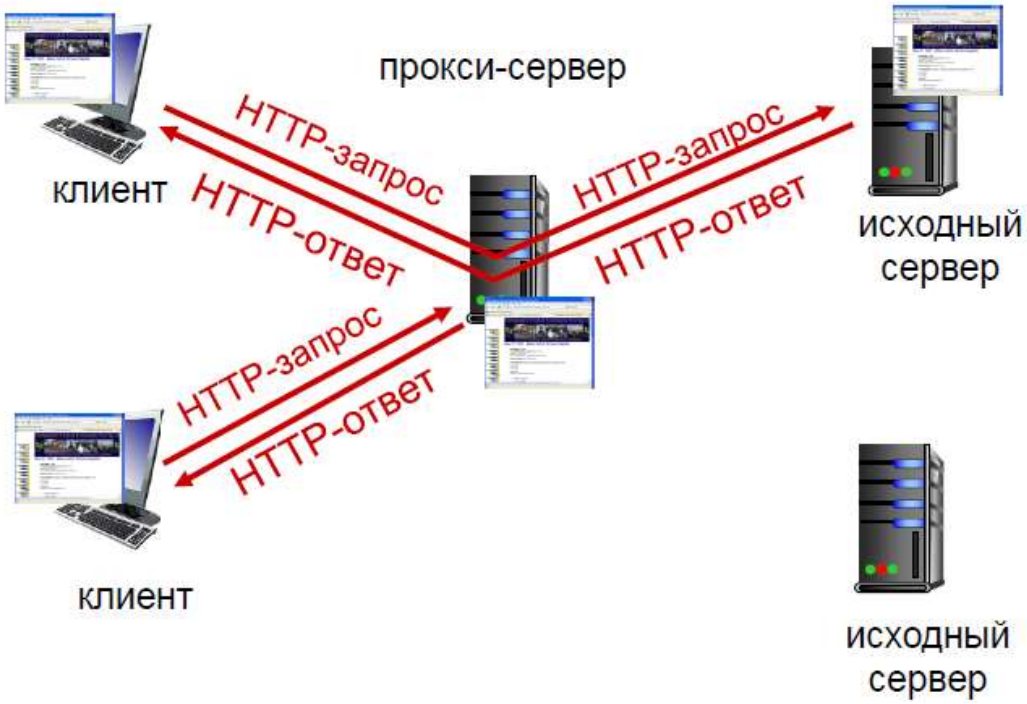
Санкт-Петербургский государственный университет

Факультет математики и компьютерных наук

Веб-кэш (прокси-сервер)

Цель: удовлетворять запросы клиента без участия исходного веб-сервера

- пользователь настраивает браузер: веб-доступ через прокси
- браузер отправляет запрос прокси-серверу
  - объект в кэше: прокси-сервер возвращает объект
  - в противном случае прокси-сервер запрашивает объект у исходного сервера и возвращает его клиенту
  - можно организовать **black list**



Веб-кэш (прокси-сервер)

- Кэш выполняет роль клиента и сервера одновременно
  - сервер для клиента, выполняющего запрос
  - клиент для исходного сервера
- часто кэш устанавливается Интернет-провайдером

Зачем нужно веб-кэширование?

- уменьшить время отклика на запросы клиентов
- уменьшить трафик в сети организации
- позволяет провайдерам эффективнее доставлять контент (как и одноранговый файлообмен)

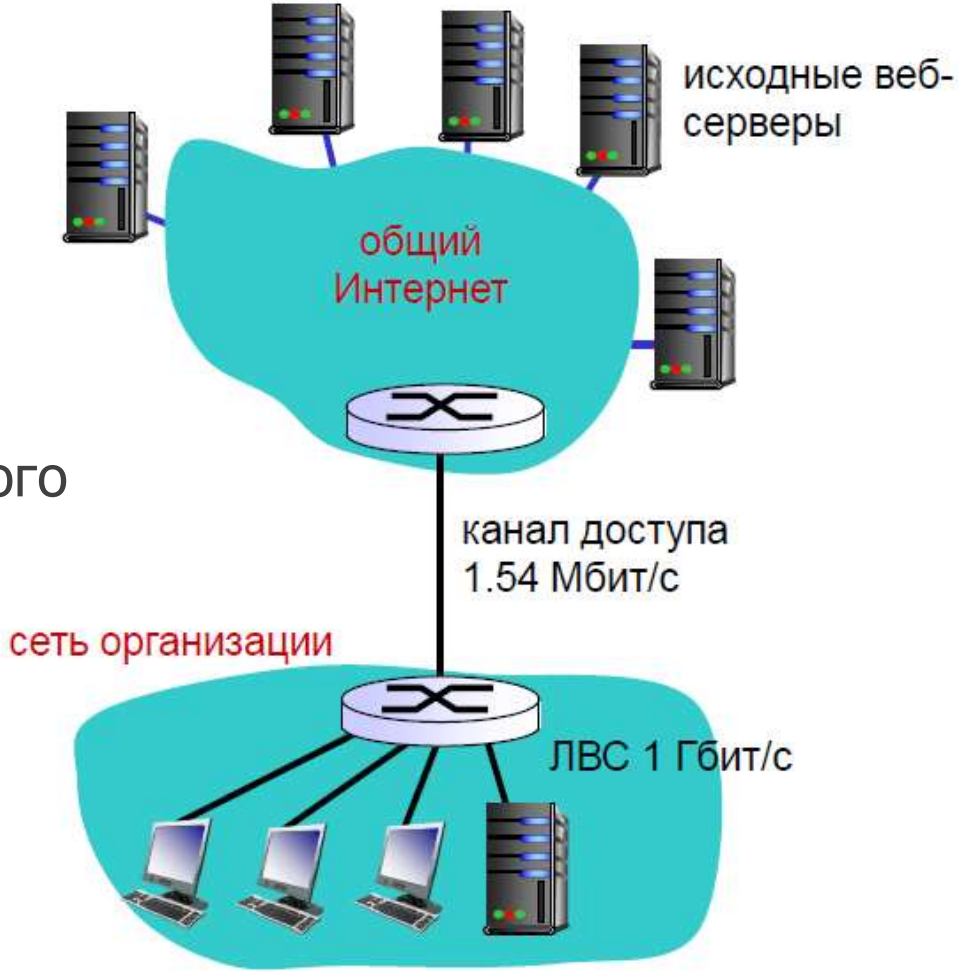
Пример кэширования

Предположения:

- средний размер объекта: 100 Кбит
- средняя частота запросов от браузеров к серверам: 15 запросов/с
- средняя скорость передачи данных к браузерам: 1,50 Мбит/с
- RTT от маршрутизатора организации до любого веб-сервера: 2 с
- пропускная способность канала: 1.54 Мбит/с

Результаты:

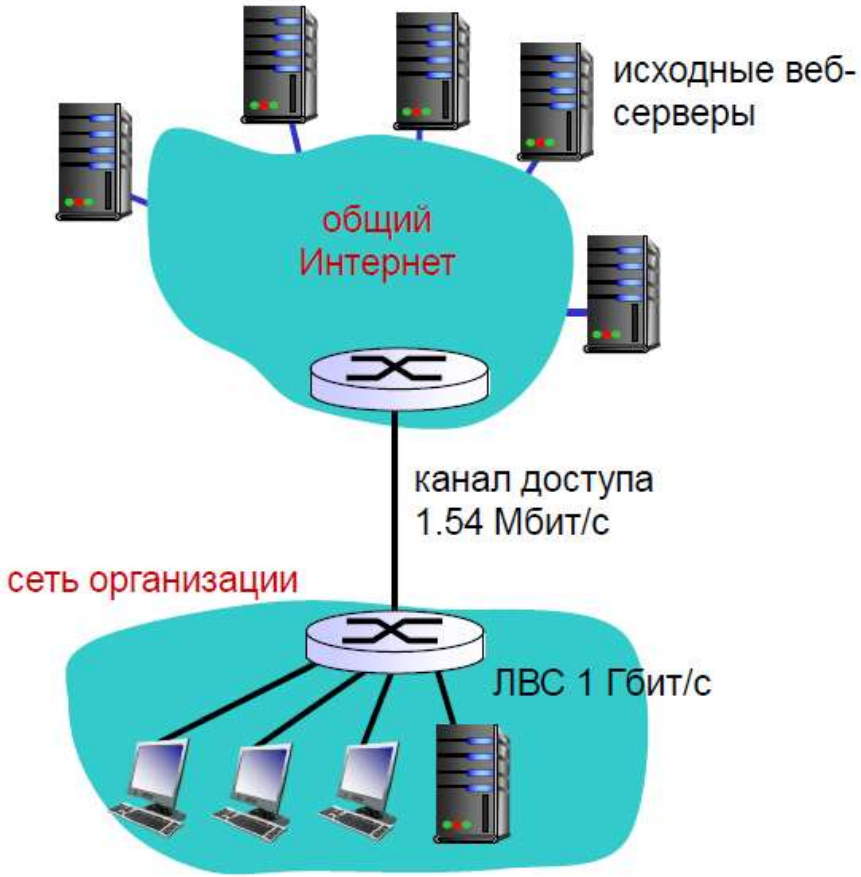
- загрузка ЛВС: 15% (=(15 запросов/с)×(100 Кбит/запрос)/(1 Гбит/с))
- загрузка канала доступа = 99%
- общая задержка = задержка Интернета + задержка доступа + задержка ЛВС = 2 с + минуты + микросекунды



Как уменьшить задержку?

Можно увеличить скорость доступа с 1.5 Мбит/с, до 10 Мбит/с.
Это понизит интенсивность трафика до 0,15, и в этом случае **общее время ответа будет примерно равно 2 с** (потому что интенсивность трафика в сети будет примерно равна интенсивности трафика внутри ЛВС) - то есть задержке Интернета

Но данный вариант решения предполагает, что сеть организации должна быть модернизирована, а это дополнительные расходы

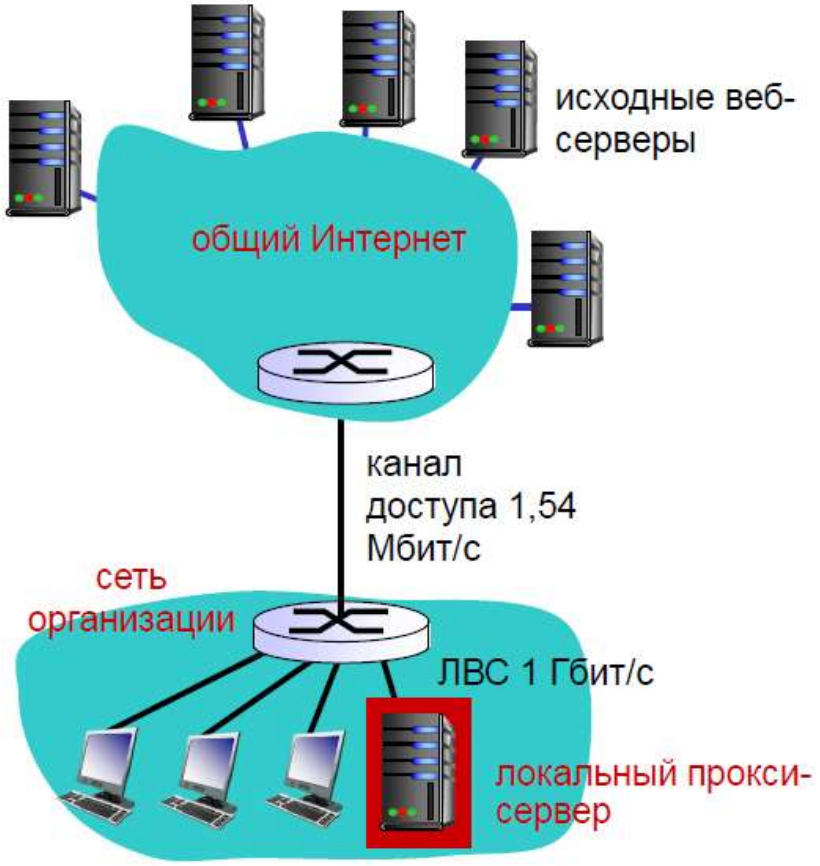


Пример кэширования: установка локального прокси

Рассмотрим альтернативное решение, в котором вместо модернизации сети используется прокси-сервер организации

Доля запросов, обслуживаемых прокси-сервером (коэффициент попадания в кэш), обычно варьируется от 0,2 до 0,7

Какова будет загрузка канала доступа ?
А общая задержка ?

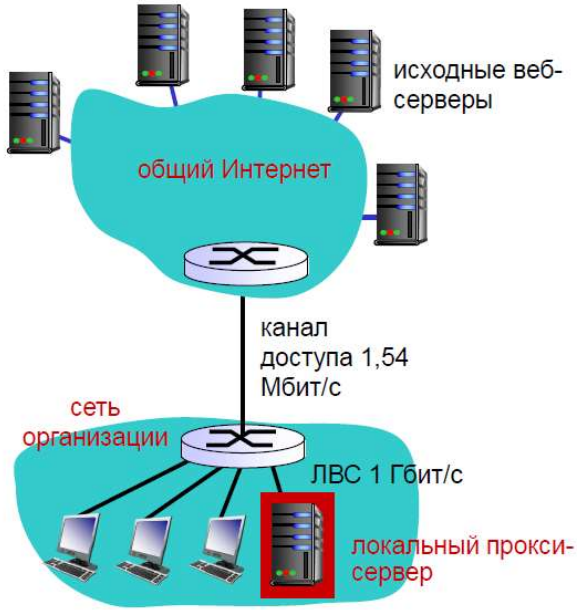


Пример кэширования: установка локального прокси

Вычисляем загрузку канала и задержку в случае с прокси:

- предполагаем, что коэффициент попадания в кэш = 0,4
  - 40% запросов обрабатываются прокси-сервером,
  - 60% - исходными веб-серверами
  - **загрузка канала доступа:**
    - 60% запросов используют канал
  - скорость данных через канал к браузерам= (0,6*15)×100Кбит/с = 0,9 Мбит/с
    - загрузка = 0,9/1,54 = 0,58
  - **общая задержка**
    - = 0,6 × (задержка от исходных серверов) + 0,4 × (задержка от прокси)
    - = 0,6 * 2 + 0,4 (~миллисекунды)
    - = ~ 1,2 секунды
    - **меньше, чем при канале в 10 Мбит/с**

  (и к тому же дешевле!)



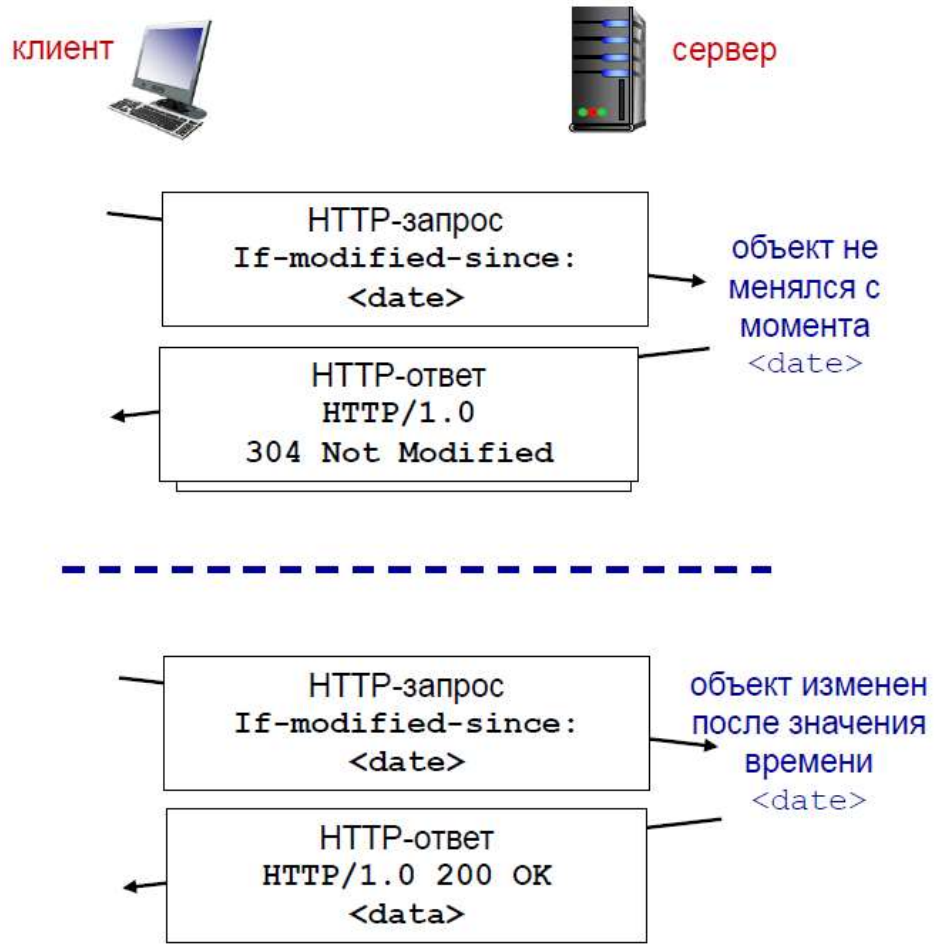
Условный GET-запрос

Цель: не отправлять объект, если в кэше существует его актуальная версия

- нет задержек передачи
- ниже загрузка канала

- **cache**: указывается дата кэшированной копии
  If-modified-since: <date>

- **server**: ответ не содержит объект, если копия в кэше актуальна:
  HTTP/1.0 304 Not Modified



Программирование сокетов UDP и TCP

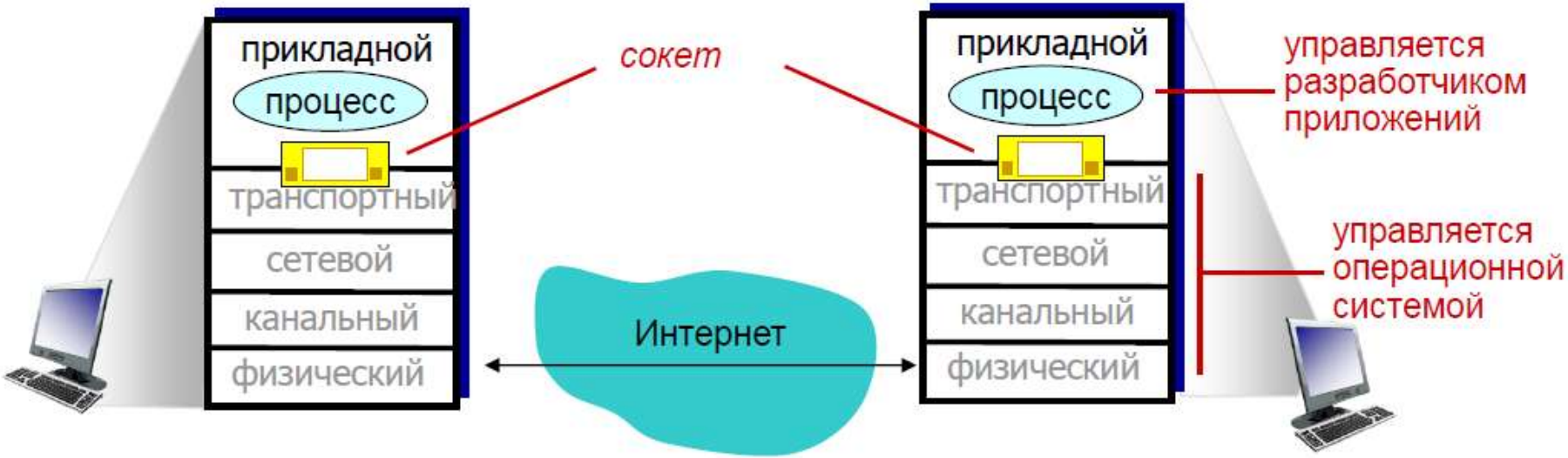
Санкт-Петербургский государственный университет

Факультет математики и компьютерных наук

Программирование сокетов

Цель: научиться разрабатывать клиент-серверные приложения, которые взаимодействуют при помощи сокетов

Сокет: дверь между процессом приложения и транспортным протоколом



Программирование сокетов

Два типа сокетов для двух транспортных служб:

- UDP: дейтаграммы без гарантии доставки
- TCP: гарантия доставки, данные в виде потока байтов

Пример приложения:

1. Клиент читает строку символов (данные) с клавиатуры и отправляет данные серверу
2. Сервер получает данные и преобразует символы в верхний регистр
3. Сервер отправляет измененные данные клиенту
4. Клиент получает измененные данные и отображает строку на своем экране

Программирование сокетов с использованием UDP

UDP: без установления соединения

- никакого рукопожатия перед отправкой данных
- отправитель прикрепляет к каждому пакету IP-адрес и номер порта хоста назначения
- получатель извлекает из принятого пакета IP-адрес и номер порта отправителя

UDP: передаваемые данные могут быть потеряны или получены не в том порядке

С точки зрения приложения:

- Протокол UDP обеспечивает ненадежную передачу сгруппированных байт (дейтаграмм) между клиентом и сервером

Взаимодействие клиента и сервера через сокеты: UDP

серверная часть

создаем сокет с портом x:
serverSocket =
socket(AF_INET,SOCK_DGRAM)

читаем дейтаграмму из
serverSocket

записываем ответ в
serverSocket
указывая адрес
и номер порта клиента

клиентская часть

создаем сокет:
clientSocket =
socket(AF_INET,SOCK_DGRAM)

создаем дейтаграмму с адресом
сервера и портом x; отправляем
ее через
clientSocket

читаем дейтаграмму из
clientSocket

закрываем
clientSocket

Пример: UDP-клиент

UDPClient на языке Python

включаем библиотеку для работы с сокетами

```
from socket import *
serverName = 'hostname'
serverPort = 12000
```

создаем сокет клиента

```
clientSocket = socket(socket.AF_INET,
 socket.SOCK_DGRAM)
```

получаем вводимые пользователем символы

```
message = raw_input('Input lowercase sentence:')
```

прикрепляем к сообщению имя и порт сервера; отправляем в сокет

```
clientSocket.sendto(message,(serverName, serverPort))
```

читаем принятые символы из сокета в переменную

```
modifiedMessage, serverAddress =
 clientSocket.recvfrom(2048)
```

выводим полученную строку и закрываем сокет

```
print modifiedMessage
clientSocket.close()
```

Пример: UDP-сервер

UDPServer на языке Python

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_DGRAM)
serverSocket.bind(('', serverPort))
print "The server is ready to receive"
while 1:
 message, clientAddress = serverSocket.recvfrom(2048)
 modifiedMessage = message.upper()
 serverSocket.sendto(modifiedMessage, clientAddress)
```

создаем сокет

связываем с сокетом порт 12000

бесконечный цикл

читаем из сокета в переменную *message*, получаем адрес и порт клиента

отправляем измененную строку обратно клиенту

Программирование сокетов с использованием TCP

Условия для обращения клиента к серверу

- серверный процесс должен быть уже запущен
- на сервере должен быть создан входной сокет (дверь) для обращения клиента

Как происходит контакт:

- создается TCP-сокет с указанием IP-адреса и номера порта серверного процесса
- создавая сокет, клиент устанавливает TCP-соединение с сервером
- при обращении клиента

TCP-сервер создает новый сокет для серверного процесса, чтобы взаимодействовать конкретно с этим клиентом

- таким образом сервер может общаться с множеством клиентов
- по номерам портов можно различать клиентов

С точки зрения приложения:

TCP обеспечивает надежную передачу упорядоченного потока байтов между сервером и клиентом

Взаимодействие клиента и сервера через сокеты: TCP

Пример: TCP-клиент

TCPClient на языке Python

```
from socket import *
serverName = 'servername'
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_STREAM)
clientSocket.connect((serverName,serverPort))
sentence = raw_input('Input lowercase sentence:')
clientSocket.send(sentence)
modifiedSentence = clientSocket.recv(1024)
print 'From Server:', modifiedSentence
clientSocket.close()
```

создаем TCP-сокет для связи с сервером по порту 12000

нет необходимости прикреплять порт и адрес сервера

Пример: TCP-сервер

TCPServer на языке Python

создаем входной TCP-сокет

сервер начинает слушать входящие TCP-запросы

бесконечный цикл

создаем новый сокет соединения для входящего запроса

читаем байты из сокета (не используя адрес , как в UDP)

закрываем соединение с этим клиентом (но *не* входной сокет)

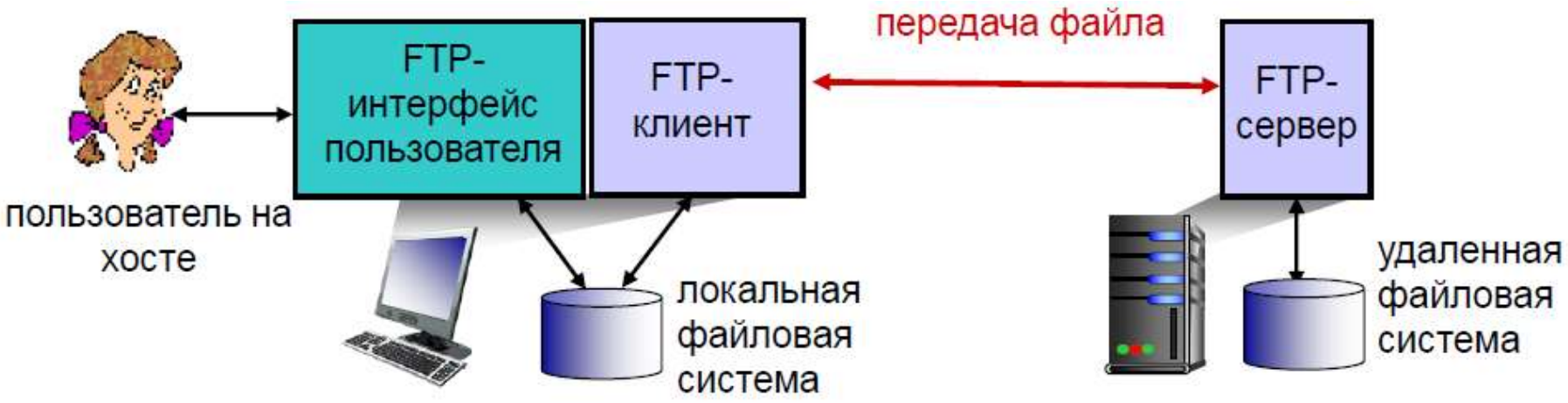
```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET,SOCK_STREAM)
serverSocket.bind(('',serverPort))
serverSocket.listen(1)
print 'The server is ready to receive'
while 1:
 connectionSocket, addr = serverSocket.accept()

 sentence = connectionSocket.recv(1024)
 capitalizedSentence = sentence.upper()
 connectionSocket.send(capitalizedSentence)
 connectionSocket.close()
```

FTP - File Transfer Protocol

Санкт-Петербургский государственный университет

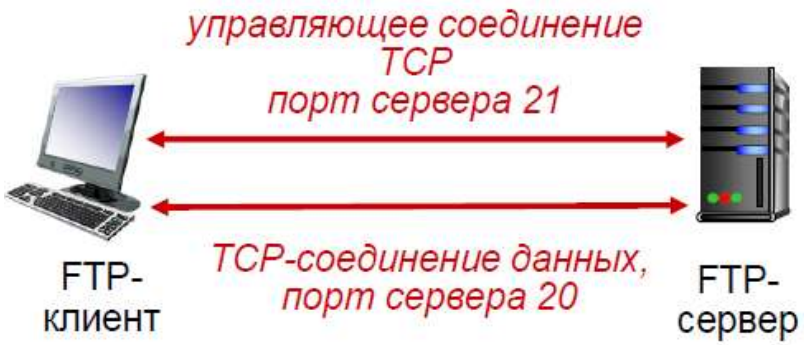
Факультет математики и компьютерных наук

FTP: протокол передачи файлов



- передача файла на удаленный хост или с него
- клиент-серверная модель
  - клиент: сторона, инициирующая передачу (на/с удаленного хоста)
  - сервер: удаленный хост
- ftp: документ RFC 959
- ftp-сервер: порт 21

FTP: отдельные соединения для передачи данных и управления



- FTP-клиент обращается к FTP-серверу по порту через TCP
- авторизация через управляющее соединение
- отправка команд и просмотр удаленного каталога через управляющее соединение
- при получении команды на передачу сервер открывает соединение данных для файла
- после передачи одного файла сервер закрывает соединение данных
- для передачи следующего файла сервер открывает еще одно соединение данных
- управляющее соединение: «вне полосы»
- FTP-сервер отслеживает состояние соединения: текущий каталог, предыдущая аутентификация

FTP-команды и ответы

Пример команд:

- отправляются в ASCII формате через управляющее соединение
- USER имя пользователя
- PASS пароль
- LIST возвращает список файлов текущего каталога
- RETR имя файла вызов (получение) файла
- STOR имя файла сохранение (отправка) файла на удаленный хост

Пример кодов ответа (документ RFC 959):

- код состояния и сообщение (как в HTTP)
- 331 Username OK, password required

(Имя пользователя корректно, нужен пароль)

- 125 data connection already open; transfer starting (Соединение открыто, обмен данными начат)
- 425 Can’t open data connection (Невозможно открыть соединение)
- 452 Error writing file (Ошибка при записи файла)

Электронная почта

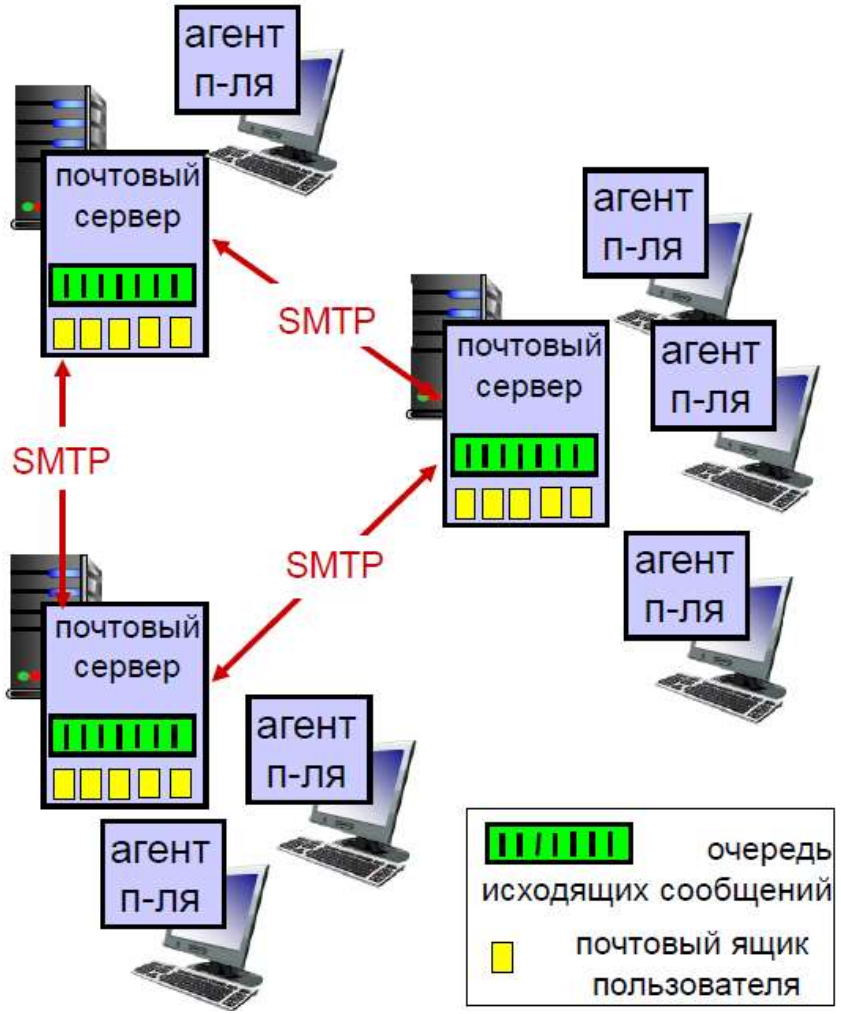
Санкт-Петербургский государственный университет

Факультет математики и компьютерных наук

Электронная почта

- Три основных компонента:
  - агенты пользователя
  - почтовые серверы
  - протокол SMTP (Simple Mail Transfer Protocol)

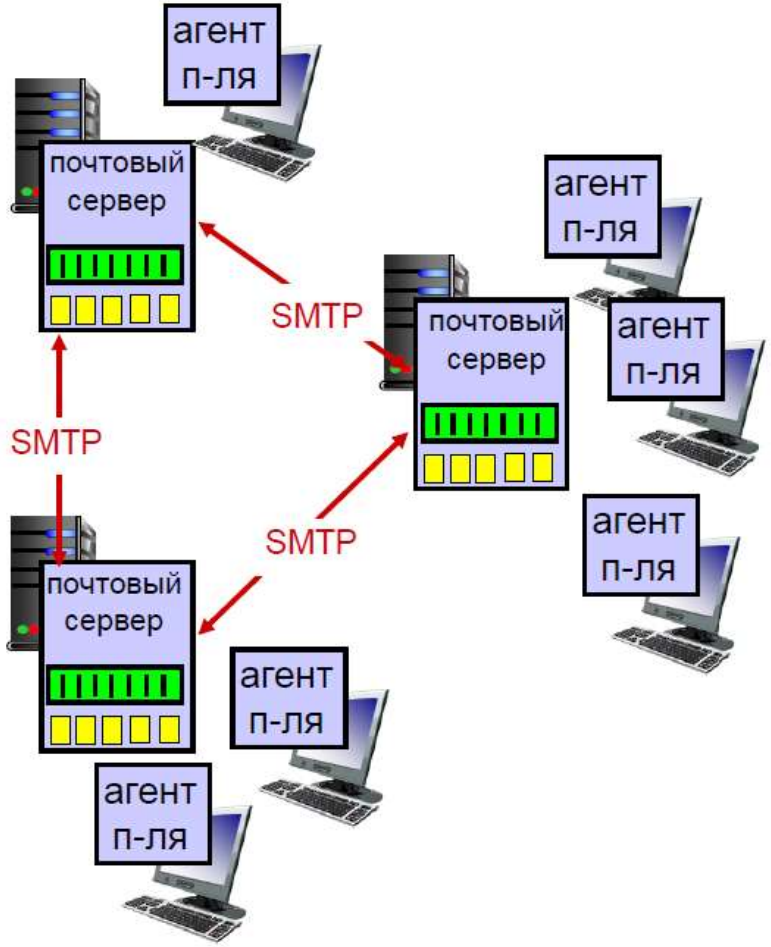
- Агенты пользователя
  - создают, редактируют, читают сообщения
  - примеры: Outlook, GMail
  - исходящие и входящие сообщения хранятся на сервере



Электронная почта: почтовые серверы

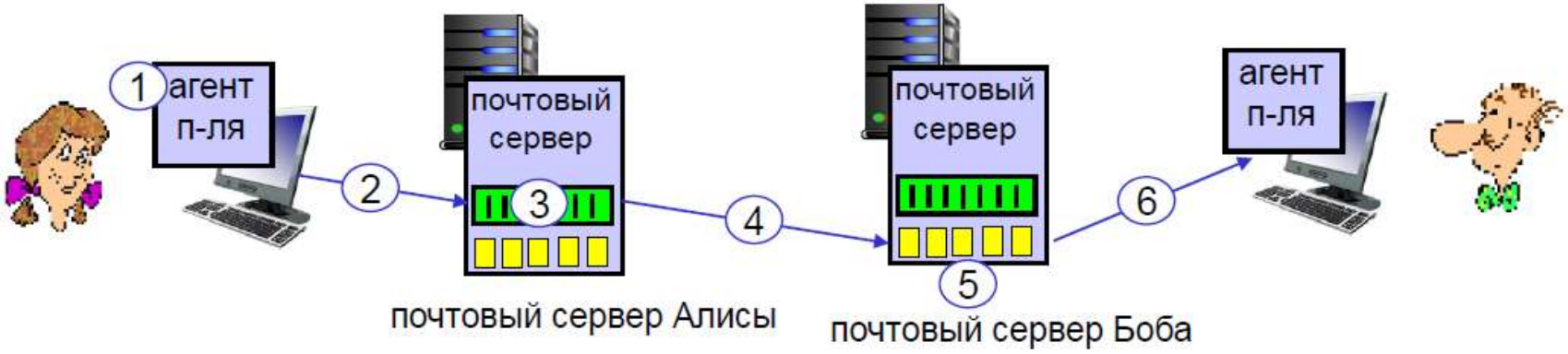
Почтовые серверы:

- почтовый ящик содержит входящие сообщения пользователя
- очередь сообщений для отправки (исходящие)
- протокол SMTP используется между почтовыми серверами для отправки сообщений
  - Обычное сообщение начинает свой путь в пользовательском агенте отправителя, доходит до почтового сервера отправителя и направляется к почтовому серверу получателя, где помещается в его почтовый ящик
  - SMTP имеет две стороны: клиентскую, исполняемую на сервере отправителя, и серверную — на сервере получателя



Сценарий: Алиса отправляет сообщение Бобу

1. Алиса использует пользовательского агента для создания сообщения для bob@someschool.edu
2. Пользовательский агент Алисы отправляет сообщение ее почтовому серверу и помещает в очередь сообщений
3. Клиентская часть SMTP открывает TCP-соединение с почтовым сервером Боба
4. Клиент SMTP отправляет сообщение через TCP-соединение
5. Почтовый сервер Боба помещает сообщение в почтовый ящик Боба
6. Боб запускает своего агента пользователя и читает сообщение



Электронная почта: протокол SMTP [RFC 5321]

- Использует TCP для надежной доставки сообщений от клиента серверу, **порт 25**
- Прямая передача: от отправляющего сервера к принимающему
- Три этапа передачи
  - рукопожатие (приветствие)
  - передача сообщений
  - закрытие
- Если есть несколько сообщений для отправки серверу, то они будут отправлены в рамках одного TCP соединения
- Взаимодействие посредством команд/ответов (подобно HTTP, FTP)
  - команды: текст в формате ASCII
  - ответы: код состояния и сообщение
- Все сообщения должны быть в 7-битном ASCII формате

Пример расшифровки сообщений, которыми обмениваются SMTP-клиент (К) и SMTP-сервер (С)

```
C: 220 hamburger.edu
K: HELO crepes.fr
C: 250 Hello crepes.fr, pleased to meet you
K: MAIL FROM: <alice@crepes.fr>
C: 250 alice@crepes.fr... Sender ok
K: RCPT TO: <bob@hamburger.edu>
C: 250 bob@hamburger.edu ... Recipient ok
K: DATA
C: 354 Enter mail, end with "." on a line by itself
K: Вы любите кетчуп?
K: Как насчет анчоусов?
K: .
C: 250 Message accepted for delivery
K: QUIT
C: 221 hamburger.edu closing connection
```

SMTP и HTTP

- SMTP использует постоянные соединения
- SMTP требует, чтобы сообщения были (тело и заголовки) в 7-битном формате ACSII
- SMTP использует комбинацию CRLF.CRLF для обозначения конца сообщения

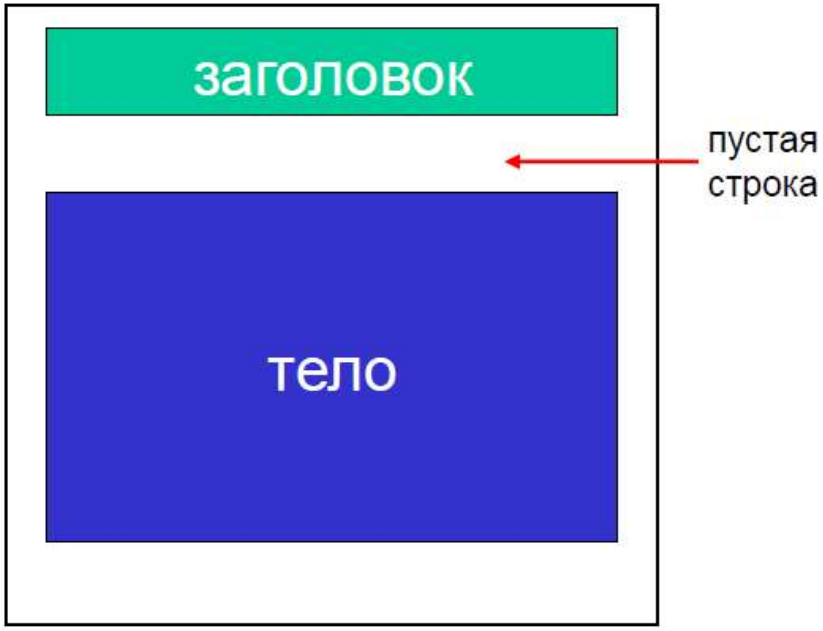
Сравнение с HTTP:

- Постоянное соединение
- HTTP: протокол получения (**pull**)
  - пользователи применяют его, чтобы получить информацию с сервера, которую туда кто-то загрузил
- SMTP: протокол отправки (**push**)
  - сервер-источник отправляет файл почтовому серверу-приемнику
- SMTP – 7-разрядный ASCII
- оба используют команды, ответы и коды состояний в формате ASCII
- HTTP: каждый объект инкапсулирован в свое собственное ответное сообщение
- SMTP: все объекты в одном сообщении

Формат сообщения электронной почты

SMTP: протокол для обмена сообщениями электронной почты
RFC 822: документ, определяющий формат почтового сообщения:

- строки заголовка:
  - To:
  - From:
  - Subject:

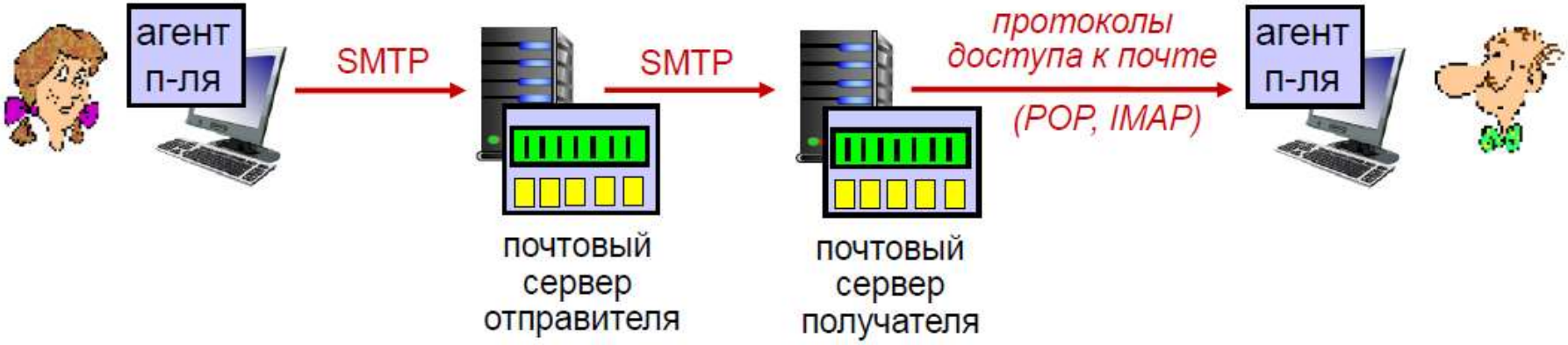


(!) Не путать с SMTP-командами MAIL FROM, RCPT TO
SMTP-команды с теми же именами составляют часть рукопожатия протокола SMTP.
А строки заголовка являются частью самого почтового сообщения

- тело: само сообщение
  - только ASCII-символы

Протоколы доступа к почте

- SMTP: доставка/хранение до сервера получателя.
- Пользовательский агент не может использовать SMTP, чтобы получать сообщения, так как SMTP — это push-протокол (протокол отправки, а не получения)
- Протокол почтового доступа: получение сообщений с сервера
  - POP: Post Office Protocol [RFC 1939]: авторизация, загрузка
  - IMAP: Internet Mail Access Protocol [RFC 1730]: расширенные функции, включая манипуляцию сообщениями на сервере
  - HTTP: Gmail, Hotmail, Yahoo! Mail, и т.д.



Протокол POP3

Фаза авторизации

- команды клиента:
  - user: имя пользователя
  - pass: пароль
- ответы сервера
  - +OK
  - -ERR

Фаза транзакции

клиент:

- list: вывод списка сообщений
- retr: вызов сообщения по номеру
- dele: удаление
- quit

```
C: +OK POP3 server ready
K: user bob
C: +OK
K: pass hungry
C: +OK user successfully logged on
```

```
K: list
C: 1 498
C: 2 912
C: .
K: retr 1
C: <содержимое сообщения 1>
C: .
K: dele 1
K: retr 2
C: <содержимое сообщения 1>
C: .
K: dele 2
K: quit
C: +OK POP3 server signing off
```

POP3 и IMAP

Еще о POP3

- в предыдущем примере использовался POP3 в режиме «загрузить-и-удалить»
  - Боб не может заново прочитать сообщение с другого места
- режим POP3 «загрузить-и-сохранить»: можно получать копии сообщения на различных клиентских местах
- POP3 не сохраняет состояние сеанса

IMAP

- все сообщения хранятся в одном месте - на сервере
- позволяет организовать хранение в каталогах
- сохраняет состояние пользовательского сеанса:
  - имена каталогов и соответствие между идентификаторами сообщений и каталогами
  - можно получать часть сообщения (например, только заголовок)

Электронная почта через веб-интерфейс

- Компания Hotmail в середине 1990-х годов
- Пользовательским агентом является веб-браузер
- Взаимодействие со своим удаленным почтовым ящиком происходит только через протокол HTTP
- POP3 и IMAP при этом не используются
- SMTP все еще используется, когда почтовый сервер клиента отправляет или получает сообщение с других почтовых серверов